An IoT-Integrated Multi-Modal Crowd Density Monitoring and Alerting System Using Heterogeneous Edge Devices

***Abstract*—Public-safety monitoring is needed in places such as transport facilities, markets, campuses, swimming-pool surroundings, and event spaces. In such settings, detecting a person is only the first step; the observed scene must also be converted into an event that can reach different low-cost devices in time. The implemented prototype uses YOLO inference with persistent person IDs, user-configurable restricted zones, a dwell-time rule for loitering, MQTT communication, ESP8266/ESP32/Arduino UNO hardware, servo and buzzer outputs, a browser dashboard, and an Amazon Alexa query interface. Because the Arduino UNO has no WiFi interface, an ESP32 is used as its serial link to the networked part of the system. Tests captured normal operation, a loitering condition, density and zone-entry information, and coordinated hardware/API status while the dashboard reported 8.5–9.2 frames/s. Manual inspection of the three supplied screenshots found 6 correctly detected people, no false person boxes, and 3 visible people that were missed. For this small sample, the corresponding values are 100% precision, 66.7% recall, 80.0% F1-score, and 1.0 person/frame MAE. These numbers are used only as an initial diagnostic check, not as benchmark accuracy. They point mainly to missed detections of small or partly submerged people. Accordingly, the paper evaluates detector behavior separately from the correctness of the complete integration and leaves deployment-level accuracy for a larger validation study.**

***Index Terms*—crowd monitoring, Internet of Things, MQTT, YOLO, edge computing, loitering detection, virtual restricted zone, accuracy evaluation, multi-modal alerting**

I. Introduction

Monitoring the number and behavior of people in sensitive or capacity-limited areas is increasingly relevant to public safety, facility management, and urban surveillance. In practice, the challenge is broader than detecting people in a camera stream. A useful deployment must transform visual observations into stable person identities, occupancy estimates, zone-based safety decisions, event logs, and physical or human-perceivable responses while operating on affordable hardware.

Computer-vision research has established both density-map approaches for crowd counting and object-detection approaches for locating people. MCNN demonstrated multi-column convolutional density estimation for scenes with large scale variation [1], while CSRNet used dilated convolution to improve dense-scene counting [2]. For real-time object-oriented monitoring, YOLO performs detection within a unified regression framework that supports efficient inference on live video [3]. SORT and Deep SORT extend frame-wise detections across time by associating them with continuing tracks, thereby enabling persistent identities and temporal analysis [4], [5].

A practical safety system must solve three coupled problems. First, the vision layer must detect and maintain identities reliably enough to support occupancy and dwell-time reasoning. Second, event logic must distinguish a short zone crossing from sustained presence such as loitering. Third, the resulting state must be delivered consistently to heterogeneous devices, including controllers that do not have native WiFi. The present work addresses these systems-level issues in one integrated prototype.

The principal contributions are:

- an end-to-end architecture integrating YOLO-based person detection, persistent track identifiers, virtual-zone logic, MQTT, a web dashboard, and heterogeneous IoT devices;
- a serial network-bridging pattern in which an ESP32 relays MQTT-derived commands to a non-WiFi Arduino UNO through UART;
- a configurable restricted-zone and dwell-time mechanism for zone-entry and loitering events;
- multi-modal notification through visual indication, physical gate/buzzer actuation, browser telemetry/event logging, and an Alexa query interface; and
- a transparent preliminary accuracy audit that separates detector performance from integration correctness and identifies missed detections as the primary current limitation.

The available evidence is interpreted within its limits. The demonstrations confirm operation from camera input to the output interfaces. The screenshot-based check provides only an initial accuracy indication; detection, tracking, event decisions, and latency require broader testing before deployment-level claims are appropriate.

II. Related Work

A. Crowd Counting and Person Detection

In highly crowded images, individual people may not be clearly separable. Crowd-counting methods therefore often estimate a spatial density distribution rather than detecting every person independently. MCNN addresses scale variation through parallel branches with different receptive fields [1], whereas CSRNet combines convolutional feature extraction with dilated convolutions [2]. Scale, occlusion, and training-data selection remain important challenges in such methods

[15]. While these approaches are effective for estimating overall crowd size, they do not retain the person-level identities needed for dwell-time monitoring. The prototype therefore uses YOLO-based person localization followed by tracking for restricted-zone analysis.

In the implemented prototype, person localization is produced by a YOLO inference engine. The YOLO stage produces class labels together with spatial bounding regions that can be used directly for real-time decisions inside predefined monitoring areas [3]. Subsequent YOLO variants have continued to balance detection quality against inference speed [7], and YOLOv7 introduced additional training and architectural changes aimed at high-throughput detection [16]. For restricted-zone monitoring, however, independent detections from each frame are not enough because dwell time belongs to a continuing person identity. SORT links detections using Kalman filtering and assignment [4]; Deep SORT strengthens that association with appearance cues [5]; and ByteTrack shows that lower-confidence detections can still help preserve trajectories through partial occlusion [8]. These tracking ideas provide the basis for retaining identity while zone residence time is accumulated.

Standard detection datasets such as MS COCO formalize large-scale object-detection evaluation [9]. For multi-object tracking, correct localization alone is not sufficient because the same person must also retain a consistent identity over time [10], [18]. CLEAR MOT provides established measures for tracking accuracy and precision [17]. HOTA extends this assessment by considering detection, association, and localization together [11]. These evaluation practices are relevant to the proposed system because occupancy, zone entry, and dwell-time alerts inherit errors from both detection and tracking. Surveillance anomaly research likewise shows that event-level evaluation should be distinguished from frame-level appearance recognition [12], within the broader anomaly-detection framework surveyed by Chandola *et al.* [19].

B. IoT Coordination and Human-Facing Alerts

MQTT is a lightweight publish-subscribe protocol standardized for machine-to-machine and IoT communication [6]. Its decoupled communication model is useful when one server must distribute the same safety state to multiple downstream consumers. This design is consistent with broader IoT architectures that separate sensing, communication, processing, and application services [13]. Edge-computing research further motivates placing time-sensitive analytics and control close to data sources to reduce unnecessary communication delay [14]. In the proposed architecture, MQTT carries monitoring state to networked microcontrollers, while an ESP32 bridges that state to the non-networked Arduino UNO.

Safety monitoring also benefits from redundant human-facing modalities. For an operator, the browser view provides the detailed scene and event history. Local hardware gives a different kind of feedback: LEDs show status, while the buzzer and servo react at the monitored site. The same current state can also be requested by voice through Alexa. In this work, Alexa is simply another way of reading the shared monitoring state rather than a contribution in voice-assistant design.

III. Problem Formulation and Proposed Solution

A. Problem Statement

Let V_t represent the camera image available at time t. Processing of that image produces the current set of people and their track identities. Each track is then compared with the selected monitoring region, its time inside the region is updated, and the loitering rule is evaluated. From the same state, the system forms the occupancy value and forwards the result to the connected endpoints. This sequence has to run continuously, including when part of the actuation hardware is a low-cost controller with no direct IP connection.

The pool-side scene is difficult for person detection because subjects may appear small, partly submerged, overlapped, or low-contrast. A missed person affects the displayed occupancy and may interrupt that track's dwell history. Recall therefore has practical importance in this application, beyond its role as an evaluation metric.

B. Proposed Processing Logic

For track i, let $(x_i(t), y_i(t))$ be the image location used for the zone test at time t. With normalized region limits (x_1, y_1, x_2, y_2), the inside/outside decision is

$$Z_i(t) = \begin{cases} 1, & x_1 \le x_i(t) \le x_2,\ y_1 \le y_i(t) \le y_2, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Whenever $Z_i(t) = 1$, the entry instant for track i is stored as s_i. Its elapsed time inside the region is then

$$\Delta_i(t) = t - s_i, \tag{2}$$

and the loitering decision becomes

$$L_i(t) = Z_i(t) \wedge [\Delta_i(t) \ge T_L]. \tag{3}$$

The variable T_L denotes the permitted dwell-time limit; the swimming-pool interface uses 5 s.

With N_t as the active-track count and C as the chosen reference capacity, the dashboard density percentage is

$$D_t = 100\frac{N_t}{C}\ \%. \tag{4}$$

After each frame is processed, count, density, alert level, and event state are distributed through MQTT and HTTP. The ESP32 converts the network state into a compact UART command for the Arduino UNO, which drives the servo and buzzer. Thus, vision processing, event logic, communication, and actuation remain separate while sharing one current state.

IV. System Architecture

A. Overview

Figure 1 shows how the implemented components are connected. Frames from a DSLR or any webcam-compatible source are sent to the central server. On the server, YOLO inference is followed by tracking, virtual-zone evaluation, loitering logic, and capacity-density calculation. The resulting

state is published through MQTT, while an HTTP API on the same server supplies the browser dashboard and Alexa skill.

The published state is consumed by an ESP8266 for LED indication, by an ESP32 that relays commands to an Arduino UNO for servo-gate and buzzer control, by the browser telemetry/event page, and by the Alexa interface for on-demand voice queries.

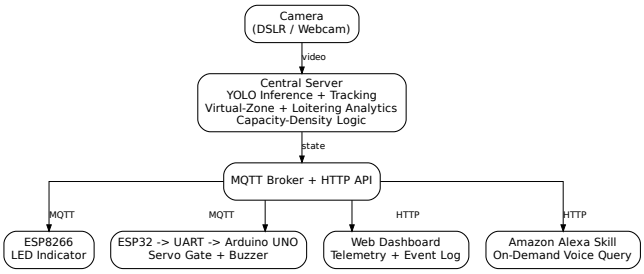


Fig. 1. Implemented monitoring architecture. Vision and event decisions are produced on the central server, and the resulting state reaches the edge actuators, browser interface, and Alexa service through MQTT or HTTP.

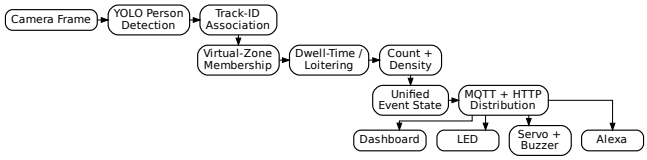


Fig. 2. Processing sequence used by the prototype, from person detection and track association through zone/dwell evaluation to distribution of the resulting alerts.

Figures 1 and 2 respectively present the deployed arrangement and the order of runtime operations. MQTT-derived commands are transferred by the ESP32 to the non-WiFi UNO, whereas person detections are first associated with continuing tracks before zone membership and dwell-time conditions are evaluated. The dashboard, actuators, and voice/API layer receive the event state only after these checks are completed. As a result, simply entering the zone for a short period does not by itself produce a loitering event.

B. Virtual Zone, Tracking, and Event State

The operator defines the monitored rectangle through a preset or normalized boundary values. When a tracked person enters it, the system records the entry and begins measuring residence time. Exceeding the selected threshold marks that track as loitering. Track continuity is therefore important because an ID switch or temporary loss can distort the accumulated dwell duration.

The interface also provides an automatic swimming-pool preset. In the supplied screenshot, the normalized limits are $x_1 = 8\%$, $y_1 = 20\%$, $x_2 = 92\%$, and $y_2 = 100\%$. Percentage coordinates make the zone independent of a specific frame resolution.

C. MQTT, Serial Bridging, and Interfaces

The server publishes application state through MQTT, keeping downstream devices independent of the vision implementation. The ESP32 receives the messages and forwards simple UART commands to the UNO, which operates the servo and buzzer. Network handling therefore stays on the WiFi-capable ESP32 while the UNO remains responsible for actuation.

The dashboard shows the live camera feed, person count, density, inference-engine label, FPS, alert state, actuator status, Alexa API status, zone controls, and event log. Alexa obtains current status from the HTTP API on request. Both interfaces therefore expose the same server-generated safety state.

V. Implementation

The server software is written in Python. OpenCV is used for frame acquisition and preprocessing, YOLO performs the deployed person inference, an MQTT client sends the current state, and FastAPI provides the HTTP endpoints used by the web page and voice integration. Eclipse Mosquitto acts as the MQTT broker. The ESP8266 and ESP32 firmware is built with the Arduino framework; the ESP32 handles the MQTT-to-serial link and the UNO executes the servo and buzzer commands.

The runtime order is illustrated in Fig. 2; both the dashboard and the physical outputs therefore receive state produced by the same frame-level decision path.

Field deployment requires valid event timestamps and, where applicable, device acknowledgements. In the supplied interface, some log entries display the timestamp as "undefined." The monitoring functions can still be demonstrated, but this field should be corrected before deployment because event reconstruction and latency analysis require a valid time reference, as does any later audit of the recorded events.

VI. Evaluation Methodology and Metrics

A. Accuracy Parameters

For manually labelled person instances, detection quality is summarized by a precision-like score, a recall-like score, and their harmonic mean. Using a for correctly matched person detections, b for spurious detections, and c for missed labelled persons, the three measures are written as

$$Q_p = \frac{a}{a+b}, \qquad Q_r = \frac{a}{a+c}, \tag{5}$$

$$Q_f = \frac{2Q_pQ_r}{Q_p+Q_r}. \tag{6}$$

Here, Q_p, Q_r, and Q_f correspond to the conventional precision, recall, and F1 measures, respectively. For occupancy, if G_k is the manually annotated number of people and $\hat{N}_k$ is the system count for sample k, the mean absolute error is

$$MAE = \frac{1}{K}\sum_{k=1}^{K}|G_k - \hat{N}_k|. \tag{7}$$

For event-level evaluation, the same precision/recall definitions can be applied to zone-entry and loitering events by treating

a correctly matched event as a true positive. End-to-end alert latency should be measured as

$$T_{e2e} = t_{actuation} - t_{event}, \tag{8}$$

with separate values for dashboard, MQTT device, buzzer/servo, and voice/API paths.

B. Preliminary Screenshot Audit

The available evidence contains three screenshots rather than an annotated video corpus. To provide an explicit accuracy parameter without fabricating benchmark data, each supplied camera view was manually inspected and visible people were counted. A system detection was treated as a true positive when a displayed person box corresponded to a visible person. No displayed person box in the three views was judged to be a false person detection. Table I summarizes the observations obtained from this limited manual assessment.

TABLE I
PRELIMINARY MANUAL AUDIT OF SUPPLIED SCREENSHOTS

View	GT	TP	FP	FN	Recall	$\lvert e \rvert$
Normal	3	1	0	2	33.3%	2
Loitering	3	3	0	0	100%	0
Zone config.	3	2	0	1	66.7%	1
Aggregate	9	6	0	3	66.7%	–

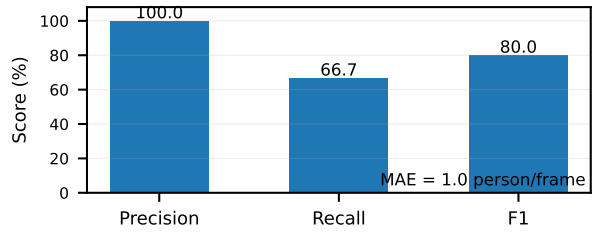


Fig. 3. Preliminary accuracy indicators obtained from the three screenshots; they are not benchmark results.

Figure 3 shows the aggregate values: 100% precision, 66.7% recall, 80.0% F1, and 1.0 person/frame MAE. Because only three screenshots were used, these are not benchmark results. They mainly reveal missed small or partly submerged people, which motivates the improvements in Section VII.

VII. RESULTS AND DISCUSSION

The integrated prototype was exercised with a live camera overlooking the pool-side scene. The available captures cover three operating situations: a normal state, a detected loitering event, and adjustment of the restricted zone. The results are therefore read in two ways: the screenshots directly show how the integrated system behaves, whereas they provide only limited evidence about the accuracy of the vision stage.

A. Normal Operating State

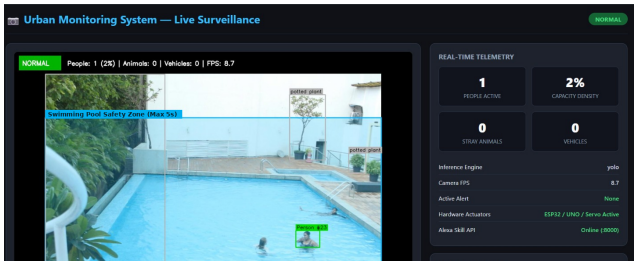


Fig. 4. Dashboard during normal operation: one active person, 2% density, 8.7 frames/s, and no active alert.

Figure 4 corresponds to the non-alert condition. The telemetry at that moment reports one active person, 2% density, and 8.7 frames/s, with the alert field set to *None*. The interface also reports the actuator connection as active and the Alexa API as online. Visual inspection of the camera frame, however, shows three people while the detector has drawn a box around only one of them. The figure therefore confirms that the monitoring and interface components are exchanging state correctly, but it also shows why strong detector-recall claims cannot be made from this example.

B. Loitering Alert State

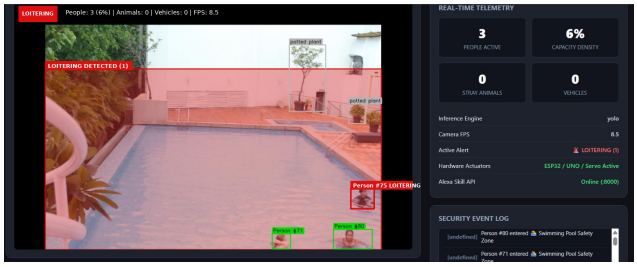


Fig. 5. Loitering alert state. Three active people and 6% capacity density are reported at 8.5 frames/s, with one tracked subject identified as loitering.

Figure 5 shows the transition to an event condition. Three people are detected and the dashboard reports 6% capacity density at 8.5 frames/s. One tracked subject is highlighted as loitering while the two other identities remain ordinary person detections. The telemetry simultaneously shows *LOITERING (1)*, and the security log records zone-entry and loitering-related events. In this sampled frame, the manual audit finds all three visible people detected. The consistency among overlay, active-alert field, and event log demonstrates successful event propagation across the monitoring interface.

TABLE II
OBSERVED PROTOTYPE TELEMETRY

State	People	Density	FPS	Active Alert
Normal	1	2%	8.7	None
Loitering	3	6%	8.5	Loitering (1)
Zone configuration	–	–	9.2	None

Across the three submitted captures, displayed camera processing rate ranges from 8.5 to 9.2 frames/s, with a mean of approximately 8.8 frames/s. This demonstrates a continuously updating interface on the tested setup. It should not yet be interpreted as a hardware-independent real-time guarantee because the processor/GPU configuration, frame resolution, confidence threshold, and long-run throughput distribution have not been documented.

C. Restricted-Zone Configuration and Event Logging

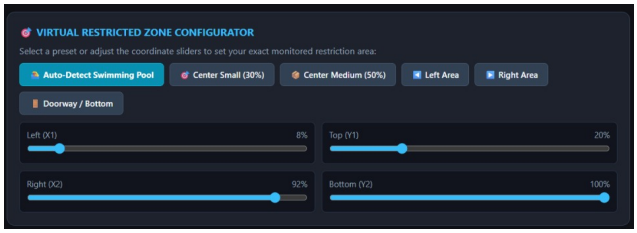


Fig. 6. Virtual restricted-zone configurator. The displayed normalized bounds are $x_1 = 8\%$, $y_1 = 20\%$, $x_2 = 92\%$, and $y_2 = 100\%$.

Figure 6 shows how the monitored area can be adjusted without modifying detection code. Presets include automatic pool selection, center, left/right, and doorway/bottom regions; sliders also adjust each boundary. The shown zone spans 8–92% of image width and 20–100% of image height. This allows the monitored area to change without moving the camera. Two person boxes are shown although three people are visible, again indicating that recall for small or overlapping people should improve before safety-critical use.

D. Useful Result: Integration Correctness Versus Vision Accuracy

The results need to be interpreted as two separate issues. One concerns whether the complete system responds consistently to whatever state is produced by the vision module. In the normal and loitering examples, the person IDs, density readings, alert indication, event records, actuator status, and API status change in a mutually consistent way. The other issue is the accuracy of the vision state itself. The present manual audit is too limited to establish that accuracy and also shows that some visible people can be missed.

This separation matters in a safety application. A person omitted by the detector cannot be recovered later by a perfectly reliable MQTT path; equally, accurate detections have little operational value if the resulting alert never reaches the physical devices. What is demonstrated here is that the downstream components receive and reflect the generated state consistently. The remaining uncertainty lies earlier in the chain, where the vision module can still miss people.

VIII. Accuracy Improvement and Validation Plan

The next technical objective is to reduce missed detections of small and partly submerged people without sacrificing the low false-positive level observed in the available samples. Four practical changes are proposed. First, fine-tune the YOLO-family detector on pool-side and surveillance imagery containing partially visible people rather than relying only on generic pretrained weights. Second, evaluate increased inference resolution and confidence/IoU threshold settings, because small targets can disappear when aggressive down-scaling or confidence filtering is used. Third, use tracking persistence to bridge short detector dropouts: an established track should not be deleted after a single missed frame, and short-term state prediction can preserve dwell continuity. Fourth, include hard negative samples such as plants, reflections, rails, and pool-side structures to prevent recall tuning from creating false alarms.

A publication-grade experiment should use multiple video sequences partitioned into training/tuning and held-out test sets. The test set should contain different lighting conditions, person sizes, partial immersion, occlusion, entries/exits, short visits, and genuine loitering cases. Results should report detector precision/recall/F1 and mAP using standard object-detection practice [9], counting MAE, track identity switches and HOTA-style tracking measures [11], zone-event precision/recall, loitering-event precision/recall at several dwell thresholds, FPS distribution, and end-to-end alert latency. A confusion matrix for event classification and an ablation comparing detection-only versus detection-plus-tracking would make the contribution substantially stronger.

TABLE III
Recommended Quantitative Validation for Final Deployment Study

Layer	Metric	Purpose
Detection	Precision, Recall, F1, mAP	Person localization accuracy
Counting	MAE	Occupancy-count error
Tracking	ID switches, track recall	Identity continuity
Zone events	Precision/Recall	Entry/exit decision correctness
Loitering	Precision/Recall	Dwell-alert correctness
Runtime	FPS, T_{e2e}	Responsiveness
IoT	Delivery/actuation success	Communication reliability

The goal of this plan is not to replace the current systems contribution but to connect it to measurable safety performance. In a final experimental campaign, the architecture should remain unchanged while detector/tracker variants are compared under the same zone and actuation logic. This isolates whether a change improves vision accuracy without altering the downstream alert mechanism.

IX. Limitations and Threats to Validity

The present accuracy figures come from only three manually inspected screenshots and are therefore preliminary rather than statistically reliable. They do not measure mAP because bounding-box ground truth and IoU matches are unavailable, nor tracking accuracy because annotated trajectories were not supplied. Manual reference counts may also be uncertain when only part of a person is visible.

The pool-side sequence is low density and does not cover dense crowds, severe occlusion, night scenes, rain, strong

glare, or major viewpoint changes. The 8.5–9.2 frames/s readings come from the live dashboard rather than a controlled throughput benchmark. Some event-log timestamps are also undefined and must be corrected before latency analysis or forensic reconstruction.

Alexa currently supports on-demand status queries and has not been evaluated for emergency notification. Future tests should cover proactive alerts, multiple zones, network recovery, long-duration device behavior, privacy-aware video handling, and user response to the available alert modes.

X. Conclusion

The implemented prototype brings the vision pipeline and IoT response path together for the monitored pool-side scenario. Person detections are generated by YOLO, while track identifiers maintain continuity across frames so that the restricted-zone module can measure how long each subject remains inside the selected area. The resulting event state is passed through MQTT to the connected edge devices. At the user-interface level, the browser displays the live telemetry, the local hardware responds through the actuator path, and Alexa can retrieve the same system state on request. Because the Arduino UNO has no native WiFi connection, the ESP32 acts as the network-to-UART bridge. During the recorded demonstrations, changes in the camera view were followed by corresponding updates in occupancy, loitering indication, event logging, hardware status, and the voice/API output. The dashboard showed a processing rate between 8.5 and 9.2 frames/s.

A manual inspection of the three available screenshots resulted in 100% snapshot precision, 66.7% recall, an 80.0% F1-score, and an MAE of 1.0 person per frame. These numbers come from a very small sample and should therefore be treated only as an initial indication of detector behavior, not as deployment accuracy. The main weakness visible in the examples is the missed detection of people who appear small or are partly submerged. The current evidence is sufficient to demonstrate the end-to-end operation of the architecture, but it is not sufficient for a safety-critical performance claim. A larger study should therefore evaluate domain-specific detector tuning, image-resolution and threshold choices, persistence of person tracks, and performance on held-out video sequences.

References

[1] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 589–597.

[2] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 1091–1100.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[4] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.

[5] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017, pp. 3645–3649.

[6] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019.

[7] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020.

[8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 1–21.

[9] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[10] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," arXiv:1603.00831, 2016.

[11] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *Int. J. Comput. Vis.*, vol. 129, pp. 548–578, 2021.

[12] W. Sultani, C. Chen, and M. Shah, "Real-world anomaly detection in surveillance videos," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6479–6488.

[13] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347–2376, 2015.

[14] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[15] V. A. Sindagi and V. M. Patel, "A survey of recent advances in CNN-based single image crowd counting and density estimation," *Pattern Recognit. Lett.*, vol. 107, pp. 3–16, May 2018.

[16] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.

[17] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP J. Image Video Process.*, vol. 2008, Art. no. 246309, 2008.

[18] P. Dendorfer *et al.*, "MOTChallenge: A benchmark for single-camera multiple target tracking," *Int. J. Comput. Vis.*, vol. 129, pp. 845–881, 2021.

[19] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, Art. no. 15, Jul. 2009.